CONCEPTS HANDBOOK

LangChain Concepts Handbook

Simple explanations, durable mental models, mini-exercises, and links to the official documentation.

Focus	Python · LangChain · LangGraph · LangSmith
Method	Simple mental models + hands-on exercises + official links
Source	Official LangChain documentation index
Edition	Python-focused · generated October 2026

How to use this PDF

Follow the sections in order, but build continuously. Every blue underlined title opens the corresponding official documentation page.

> Based on the current documentation available when generated. LangChain evolves quickly; use the linked changelog and documentation indexes to check updates.

Contents

Part I — The Core Agent Loop 5

1. Models, Messages, and Tools . . . 5

1.1 Models: the decision maker . . . 5

Mental model . . . 5

Cover . . . 6

Official docs . . . 6

1.2 Messages: the protocol and history . . . 6

Mental model . . . 6

Cover . . . 6

Official docs . . . 6

1.3 Tools: controlled capabilities . . . 6

Mental model . . . 6

Mini-exercise . . . 7

Official docs . . . 7

2. The Agent Loop . . . 7

Mental model . . . 7

Cover . . . 8

Mini-exercise . . . 8

Official docs . . . 8

3. Structured Output . . . 8

Mental model . . . 8

Cover . . . 8

Mini-exercise . . . 9

Official docs . . . 9

Part II — State, Memory, and Live Execution 9

4. Memory . . . 9

Mental model . . . 9

4.1 Short-term memory . . . 9

4.2 Long-term memory . . . 9

Mini-exercise . . . 10

Official docs . . . 10

5. Streaming . . . 10

Mental model . . . 10

Cover . . . 10

Mini-exercise . . . 10

Official docs . . . 11

Part III — Controlling the Loop 11

6. Middleware . . . 11

Mental model . . . 11

Mini-exercise . . . 11

Official docs . . . 12

7. Context Engineering . . . 12
Mental model . . . 12
Mini-exercise . . . 12
Official docs . . . 12
8. Guardrails and Human-in-the-Loop . . . 13
Mental model . . . 13
Mini-exercise . . . 13
Official docs . . . 13
Part IV — Retrieval and RAG **13**
9. Retrieval-Augmented Generation . . . 13
Mental model . . . 13
RAG patterns . . . 14
Mini-exercise . . . 14
Official docs . . . 14
Part V — LangGraph **14**
10. State, Nodes, and Edges . . . 14
Mental model . . . 14
State . . . 15
Nodes . . . 15
Edges . . . 15
Mini-exercise . . . 15
Official docs . . . 15
11. Checkpoints and Durable Execution . . . 16
Mental model . . . 16
Mini-exercise . . . 16
Official docs . . . 16
12. Interrupts . . . 16
Mental model . . . 16
Mini-exercise . . . 17
Official docs . . . 17
Part VI — Multi-Agent Systems **17**
13. Multi-Agent Patterns . . . 17
Mental model . . . 17
Main patterns . . . 17
Mini-exercise . . . 17
Official docs . . . 18
Part VII — LangSmith **18**
14. Tracing and Observability . . . 18
Mental model . . . 18
Mini-exercise . . . 18
Official docs . . . 19
15. Evaluation . . . 19
Mental model . . . 19

Offline evaluation . . . 19
Online evaluation . . . 19
Mini-exercise . . . 20
Official docs . . . 20
16. Deployment . . . 20
Mental model . . . 20
Mini-exercise . . . 20
Official docs . . . 21
Suggested Learning Project **21**
Official documentation indexes . . . 21

> Goal: Build durable mental models for LangChain, LangGraph, and LangSmith—not memorize APIs. Core picture: LangChain builds the agent, LangGraph runs the stateful workflow underneath it, and LangSmith helps you observe, test, and deploy it.

0. The Whole System in One Picture

```
User
  ↓
Messages → Model → Tool request → Tool execution
              ↑            ↓
              └── Tool result ──┘
                     repeat
                       ↓
                  Final response

Around the loop:
- State and memory preserve information
- Middleware changes or observes behavior
- Guardrails and humans control risky actions
- Streaming exposes work as it happens
- LangGraph provides durable execution
- LangSmith traces, evaluates, and deploys the system
```

Mental model

An agent is a model inside a software harness.

- The model chooses what to say or which tool to request.
- The harness owns the loop, tools, prompts, state, middleware, and stopping rules.
- LangChain's `create_agent` supplies a standard harness.
- LangChain agents run on LangGraph, so they inherit graph execution, persistence, streaming, and interrupts.

Mini-exercise

Draw a box around everything the model does. Then list everything outside that box. The outside list—tool execution, retries, memory, permissions, logging—is the agent harness.

Official docs

- LangChain overview
- Agents
- LangGraph overview

Part I — The Core Agent Loop

1. Models, Messages, and Tools

1.1 Models: the decision maker

Mental model

A chat model is a decision engine with a message interface. It receives messages and returns an AI message. That message may contain:

- Text for the user
- Structured content
- One or more tool-call requests
- Metadata such as token usage

The model proposes actions; it does not execute Python functions itself.

Cover

- Initializing provider-specific chat models
- Invoking a model directly
- Binding tools
- Model configuration versus application logic
- Why provider packages remain separate from `langchain`

Official docs

- Models
- Install LangChain

1.2 Messages: the protocol and history

Mental model

Messages are the typed event log shared with the model.

Common roles:

- System message: instructions and operating policy
- Human message: user input
- AI message: model output, possibly including tool calls
- Tool message: the result of a specific tool call

A tool result must be linked to the tool call that requested it. Think of the tool-call ID as a correlation ID in a distributed system.

Messages are not identical to memory:

- Messages are one possible part of state.
- Memory decides which information persists and for how long.
- Context engineering decides which messages the model sees on a particular call.

Cover

- Message roles
- Text and multimodal content blocks
- Tool calls and tool results
- Message metadata and artifacts
- Message history versus the model's current context window

Official docs

- Messages

1.3 Tools: controlled capabilities

Mental model

A tool is a typed capability boundary:

```
name + description + input schema + implementation + result
```

The description and schema tell the model when and how to request the tool. The implementation remains ordinary application code.

Good tools should be:

- Narrow in purpose
- Clearly named
- Explicit about inputs
- Predictable in outputs
- Safe to retry when possible
- Protected when they have side effects

Tool design is interface design for two callers: the model that selects the tool and the code that executes it.

Mini-exercise

Create three tiny tools:

1. `get_weather(city: str)` — read-only
2. `search_orders(customer_id: str)` — reads private data
3. `cancel_order(order_id: str)` — changes external state

For each tool, write:

- A one-sentence description
- Its input schema
- Whether retrying is safe
- Whether human approval is required

Official docs

- Tools

---

2. The Agent Loop

Mental model

The standard loop is:

1. Send messages and available tools to the model.
2. The model returns either:
   - a final response, or
   - tool calls.
3. The harness executes requested tools.
4. Tool results become tool messages.
5. The model sees the updated conversation.
6. Repeat until the model stops requesting tools.

In simplified Python:

```
while True:
    ai_message = model.invoke(messages)

    if not ai_message.tool_calls:
        return ai_message
```

```
    messages.append(ai_message)

    for call in ai_message.tool_calls:
        result = execute_tool(call)
        messages.append(tool_result_message(call, result))
```

The key insight is that an agent is not a mysterious new kind of model. It is a loop around a model.

Cover

- `create_agent`
- The model node and tool node
- Multiple or parallel tool calls
- Stopping conditions
- Step and recursion limits
- Tool errors
- Why the same loop can be represented as a graph

Mini-exercise

Implement the loop manually with one calculator-like tool. Log:

- Every model call
- Every requested tool
- Every tool result
- The final response

Then rebuild it with `create_agent` and compare the event sequence.

Official docs

- Agents
- Context engineering: the agent loop
- LangGraph Graph API

---

3. Structured Output

Mental model

Structured output is an API contract for model output, not merely a prompt asking for JSON.

Instead of parsing prose, define a schema such as a Pydantic model. LangChain asks the model for that shape, validates the result, and exposes it as structured application data.

Two broad strategies are available:

- Provider strategy: use a model provider's native structured-output support.
- Tool strategy: represent the desired response as a tool-like schema.

With agents, validated data is returned in the `structured_response` state key.

Cover

- Pydantic models, dataclasses, `TypedDict`, and JSON Schema
- Required versus optional fields
- Field descriptions

- Validation failures and retries
- Provider-native versus tool-based output
- Why structured output is useful at application boundaries

Mini-exercise

Define:

```
class SupportTicket(BaseModel):
    category: Literal["billing", "technical", "account"]
    urgency: Literal["low", "medium", "high"]
    summary: str
```

Test it with:

1. A clear request
2. An ambiguous request
3. A request missing urgency information

Decide whether missing information should produce `None`, a clarification question, or a validation retry.

Official docs

- Structured output

Part II — State, Memory, and Live Execution

4. Memory

Mental model

“Memory” is not one feature. It is information stored at different scopes.

Kind	Scope	Mental model	Typical implementation
Current model context	One model call	What the model sees now	Selected prompt and messages
Short-term memory	One thread	The conversation’s working state	Graph state plus checkpointer
Long-term memory	Across threads	Durable user/application knowledge	LangGraph store

4.1 Short-term memory

Short-term memory preserves state within a thread. It commonly contains message history, but it can also contain fields such as uploaded files, authentication status, or intermediate results.

Long histories still need management:

- Trim old messages
- Summarize older sections
- Remove large tool outputs
- Keep only task-relevant facts

4.2 Long-term memory

Long-term memory stores information independently of one conversation:

- User preferences
- Saved facts

- Prior decisions
- Learned instructions

A checkpointer saves graph snapshots for one thread. A store holds application-defined data across threads. Do not treat them as interchangeable.

Mini-exercise

Build an assistant that:

1. Remembers the current conversation under `thread_id="conversation-1"`.
2. Starts a fresh conversation under another thread ID.
3. Stores the user's preferred language in a long-term store.
4. Recalls that preference in both threads.

Official docs

- Short-term memory
- Long-term memory
- Memory concepts
- LangGraph persistence
- LangGraph stores

---

5. Streaming

Mental model

Streaming is an observation channel over an ongoing run. It does not change the workflow's logic.

Different consumers need different views:

- A chat UI needs token or message chunks.
- A progress panel needs node or agent updates.
- A debugger needs state snapshots and tool events.
- Application code may emit custom progress updates.

Do not reduce streaming to "print tokens sooner." Agent streaming can reveal the whole execution process.

Cover

- Model token streaming
- Agent progress
- Tool-call chunks
- State snapshots
- Custom updates
- Final output after the stream completes
- Streaming across interrupts
- Avoiding sensitive-data leaks in partial output

Mini-exercise

Stream one agent run and render three separate channels:

1. Assistant text
2. Tool name and arguments
3. Current workflow step

Confirm that combining or hiding channels does not change the final agent result.

Official docs

- LangChain streaming
- LangChain event streaming
- LangGraph streaming
- LangGraph event streaming

Part III — Controlling the Loop

6. Middleware

Mental model

Middleware is cross-cutting code placed around stages of the agent loop.

It can observe or change:

- The prompt before a model call
- Which tools the model sees
- Model requests and responses
- Tool calls and results
- State before or after a step
- Retry, fallback, logging, and termination behavior

Use middleware for policies that apply across the loop. Keep core business operations in tools and workflow nodes.

Typical uses:

- Logging and metrics
- Dynamic prompts
- Retries and fallbacks
- Summarization
- PII handling
- Rate limiting
- Tool restrictions
- Human approval

Mini-exercise

Add middleware that:

1. Records each requested tool name.
2. Removes one dangerous tool for untrusted users.
3. Summarizes history after a chosen message count.

Write down which changes are transient for one model call and which permanently update state.

Official docs

- Middleware overview
- Prebuilt middleware
- Custom middleware

---

7. Context Engineering

Mental model

Context engineering means supplying the right information and tools, in the right format, at the right moment.

More context is not automatically better. Irrelevant context consumes tokens, hides important instructions, and can reduce reliability.

Separate four questions:

1. What should the model see now?
2. What should tools be able to access?
3. What should persist in thread state?
4. What should persist across conversations?

Important data locations:

Location	Use
Model context	Temporary prompt, selected messages, tools, response schema
Runtime context	User ID, permissions, clients, configuration, dependencies
State	Mutable, thread-scoped working data
Store	Durable, cross-thread application memory

Runtime context is dependency injection. It is not the same thing as the model's token context window.

Mini-exercise

Given a customer-support agent, place each item in the correct location:

- Database connection
- Customer ID
- Current conversation
- Retrieved refund policy
- Preferred language
- Temporary tool restriction
- Unresolved ticket number

Then explain which items the model actually needs to see.

Official docs

- Context engineering in agents
- Context concepts

---

8. Guardrails and Human-in-the-Loop

Mental model

Guardrails answer: “May this input, output, or action proceed?”

Two common forms:

- Deterministic checks: schemas, allowlists, regex, permissions, limits
- Model-based checks: classifiers or judge models for nuanced policies

Prefer deterministic checks when a rule can be stated precisely.

Human-in-the-loop is a special control path for proposed actions. The workflow pauses before the action, persists its position, and waits for a decision.

Typical decisions:

- Approve: run the proposed tool call
- Edit: change arguments, then run it
- Reject: do not run it; return feedback
- Respond: supply a human answer instead of executing an “ask user” tool

HITL protects actions, while ordinary output review protects responses.

Mini-exercise

Create policies for:

- `search_docs`: automatic
- `read_customer_record`: automatic only for authorized users
- `send_email`: require approval
- `delete_account`: require approval and forbid argument editing
- User text containing an email address: redact before tracing

Test approve, edit, and reject paths.

Official docs

- Guardrails
- Human-in-the-loop
- Human-in-the-loop concepts

Part IV — Retrieval and RAG

9. Retrieval-Augmented Generation

Mental model

Retrieval is search at run time. RAG is retrieval followed by generation using the retrieved evidence.

It addresses two model limits:

- The model cannot fit an entire knowledge base into one prompt.
- The model’s training knowledge may be missing, private, or outdated.

A common indexing path is:

```
Sources → Documents → Chunks → Embeddings → Vector store
```

A common query path is:

```
Question → Search → Relevant chunks → Model context → Grounded answer
```

Retrieval is not memory:

- Retrieval searches an external knowledge source.
- Memory preserves information about prior runs, users, or application state.

RAG patterns

Two-step RAG

Always retrieve first, then generate.

- Predictable
- Easier to test
- Stable latency
- Good default when every question needs the same knowledge base

Agentic RAG

Expose retrieval as a tool; the agent decides whether and how to search.

- More flexible
- Useful with multiple sources
- Harder to predict and evaluate
- May make unnecessary or poorly formed searches

Start with two-step RAG unless dynamic retrieval decisions add clear value.

Mini-exercise

Index five short documents. Ask:

1. A question directly answered by one document
2. A question requiring two documents
3. A question the collection cannot answer

Inspect retrieved chunks separately from the final answer. Add a rule requiring the assistant to say when the retrieved evidence is insufficient.

Official docs

- Retrieval
- Build a semantic search engine
- Build a custom RAG agent with LangGraph
- RAG with Deep Agents

Part V — LangGraph

10. State, Nodes, and Edges

Mental model

A LangGraph workflow is a state machine whose steps can include models, tools, ordinary Python, or complete agents.

```
State = shared working record
Nodes = work
Edges = control flow
Reducers = rules for merging updates
```

State

State is the workflow's typed shared record. Nodes receive state and return partial updates.

A state schema should describe business facts, not merely implementation details:

```
class State(TypedDict):
    question: str
    documents: list[str]
    answer: str
```

When multiple updates target the same field, reducers define whether to replace, append, merge, or otherwise combine them.

Nodes

A node is a unit of work:

- Call a model
- Run deterministic Python
- Query a retriever
- Call an API
- Invoke another agent or subgraph

Nodes should usually return updates rather than mutating hidden global data.

Edges

Edges define what runs next:

- Fixed edge: always go to the same node
- Conditional edge: inspect state and choose a route
- Loop: return to an earlier node
- Fan-out: send work to several nodes
- `START` and `END`: graph boundaries

Mini-exercise

Build this graph without an agent:

```
START → classify question
          ├─ factual → retrieve → answer
          └─ casual  → answer directly
                         ↓
                        END
```

Record every field each node reads and writes.

Official docs

- Graph API overview
- Use the Graph API
- Choosing between Graph and Functional APIs

11. Checkpoints and Durable Execution

Mental model

A checkpoint is a saved snapshot of graph state at a workflow boundary.

A checkpointer organizes snapshots by thread. The thread ID is the cursor that tells LangGraph which execution history to load.

Checkpoints enable:

- Conversation continuity
- Recovery after failure
- Human approval that may arrive later
- State inspection
- Replay and time travel
- Durable, long-running workflows

A checkpoint does not make an external side effect repeat-safe. Email sending, billing, and deletion still need idempotency keys or other application-level protections.

Mini-exercise

Run a three-node graph with a checkpointer:

1. Stop it after the second step.
2. Inspect its state history.
3. Resume with the same thread ID.
4. Invoke it with a new thread ID.
5. Explain why the two results differ.

Official docs

- Persistence
- Checkpointers
- Memory
- Time travel

---

12. Interrupts

Mental model

An interrupt is a durable pause that asks the outside world for information or approval.

When a node calls `interrupt()`:

1. The graph saves its state through the checkpointer.
2. Execution pauses.
3. The interrupt payload is returned to the caller.
4. The caller later resumes using `Command`.
5. The same thread ID reconnects the response to the paused execution.

Design code around an interrupt so replay is safe. Work before the interrupt may be revisited, depending on the API and checkpoint boundary.

Mini-exercise

Create a purchase workflow:

```
calculate total → interrupt for approval → place order
```

Pause it, exit the Python process, then resume using the same thread ID. Add a unique order key so resuming twice cannot place two orders.

Official docs

- Interrupts
- Human-in-the-loop overview

Part VI — Multi-Agent Systems

13. Multi-Agent Patterns

Mental model

A multi-agent system is a context and responsibility design, not simply “more agents.”

Use multiple agents when specialization improves:

- Tool selection
- Prompt clarity
- Permission boundaries
- Parallelism
- Context isolation

Do not add agents merely to imitate an organization chart. A single agent with better tools or middleware is often simpler.

Main patterns

Pattern	Mental model	Best fit
Subagents	A supervisor calls specialists as tools	Central control and context isolation
Handoffs	Control moves to a different specialist	Multi-stage conversations or ownership changes
Router	A classifier selects one or more workers	Clear input categories or parallel dispatch
Skills	One agent loads specialist instructions on demand	Specialization without separate agent state
Custom workflow	LangGraph explicitly controls all steps	Mixed deterministic and agentic orchestration

Context passed between agents should be deliberately filtered. Sending every specialist the full history defeats context isolation.

Mini-exercise

Implement the same support system in two ways:

1. One agent with billing and technical-support tools
2. A supervisor with billing and technical subagents

Compare:

- Prompt size

- Number of model calls
- Tool-selection accuracy
- Information exposed to each specialist
- Ease of tracing failures

Keep the multi-agent version only if it produces a measurable benefit.

Official docs

- Multi-agent overview
- Subagents
- Handoffs
- Router
- Custom workflows
- LangGraph subgraphs

Part VII — LangSmith

14. Tracing and Observability

Mental model

LangSmith tracing is the agent's flight recorder.

- A run is one unit of work, similar to a span.
- A trace is the tree of runs created by one operation.
- A thread links traces from a multi-turn session.
- A trajectory flattens a session into the ordered messages and actions taken.

A trace should answer:

- What input arrived?
- What prompt and model were used?
- Which tools were requested?
- What did the tools return?
- Where was time spent?
- Why did the final result fail?

Tracing records behavior. It does not prove that behavior is good; evaluation does that.

Mini-exercise

Trace a RAG agent request. Locate:

1. The root run
2. The retrieval run
3. Retrieved documents
4. The model call
5. The final answer
6. Latency and token information

Then attach metadata for application version and environment.

Official docs

- LangSmith Observability
- Observability concepts
- Tracing quickstart
- Trace LangChain applications
- View traces

---

15. Evaluation

Mental model

Evaluation turns “this seems good” into repeatable evidence.

Start by defining what good means for each component:

- Correct retrieval
- Correct tool selection
- Valid tool arguments
- Safe trajectory
- Accurate final answer
- Required output format

Offline evaluation

Run before deployment against curated examples.

Use it for:

- Comparing prompts or models
- Regression testing
- Benchmarking
- Testing known failure cases

Online evaluation

Run against production traces.

Use it for:

- Quality monitoring
- Safety checks
- Anomaly detection
- Discovering new test cases

The improvement loop is:

```
Production trace
  → find failure
  → add it to a dataset
  → reproduce with offline evaluation
  → fix and compare
  → deploy
  → monitor again
```

Prefer deterministic evaluators for exact requirements. Use human or LLM-based evaluators for subjective qualities.

Mini-exercise

Create ten examples for a support agent. Add evaluators for:

1. Output schema validity
2. Correct tool selection
3. Whether an unauthorized action was attempted
4. Answer helpfulness

Run two prompt versions and compare results instead of choosing by anecdote.

Official docs

- LangSmith Evaluation
- Evaluation concepts
- Evaluation quickstart
- Evaluate an LLM application
- Manage datasets

---

16. Deployment

Mental model

LangSmith Deployment is the production workflow runtime, while LangSmith Observability and Evaluation measure what the application does.

Deployment handles agent-specific runtime concerns such as:

- Durable execution
- Threads and runs
- Persistence
- Streaming
- Background work
- Horizontal scaling
- Agent Server APIs
- Cloud, hybrid, and self-hosted operation

Its main execution concepts are:

- Assistant: configured agent or graph
- Thread: persistent state container
- Run: one execution against an assistant and, optionally, a thread

Deployment is not the end of the lifecycle. A production system should continuously feed traces into monitoring and evaluation.

Mini-exercise

Prepare a minimal graph for deployment:

1. Define the graph entry point.
2. Declare dependencies.

3. Run it locally.
4. Create two runs on the same thread.
5. Stream one run.
6. Verify that traces appear in LangSmith.
7. Add one failed production trace to an evaluation dataset.

Official docs

- LangSmith Deployment
- Deployment quickstart
- Agent Server overview
- Application structure
- Deploy to Cloud
- LangGraph deployment

Suggested Learning Project

Build one small support assistant in seven passes:

1. Model and messages: answer without tools.
2. Tool loop: add order lookup.
3. Structured output: return a typed support ticket.
4. Memory: preserve a conversation and user preferences.
5. RAG: retrieve product and refund policies.
6. Safety: require approval before changing an order.
7. Production loop: trace it, create an evaluation dataset, compare two versions, and prepare it for deployment.

After each pass, answer:

- What state exists?
- What does the model see?
- What can the model request?
- What can actually cause side effects?
- What survives the current call?
- How would I inspect a failure?
- How would I prove the new version is better?

Official documentation indexes

- Complete LangChain documentation index
- Current Python build documentation index

Generated from the official LangChain documentation index and linked official pages. Links are intended for further exploration.